Table des matières

nirs4all preflight **2**

Spécification de nirs4all preflight **3**
1 Objectifs et périmètre . . . 3
2 Architecture et responsabilités . . . 3
3 Données et unités statistiques . . . 4
4 Vues spectrales et garde contre la fuite . . . 5
5 Protocoles analytiques . . . 6
6 Contrat de résultat et moteur de règles . . . 10
7 API et bindings . . . 11
8 Fournisseurs numériques et WASM . . . 12
9 Page web et composants partagés . . . 13
10 Budgets et performances . . . 14
11 Validation et livraison . . . 14
12 Décisions et limites restantes . . . 15

Roadmap d'implémentation de nirs4all preflight **16**
1 Lots et critères d'acceptation . . . 16
2 Jalons et organisation . . . 17
3 Travail amont et dépendances . . . 18
4 Risques et décisions de repli . . . 19
5 Qualification et clôture des lots . . . 20

Audit de l'intégration de nirs4all preflight **21**
Sources et périmètre . . . 21
Responsabilités et réutilisation . . . 21
Points de code confirmés . . . 22
Révisions locales inspectées . . . 22
Références scientifiques externes . . . 23

Décisions issues des reviews de nirs4all preflight **24**
1 Review du document source . . . 24
2 Review finale Astra . . . 24
3 Review finale Opus . . . 25
4 Limites de la clôture documentaire . . . 26

nirs4all preflight

Spécification et roadmap d’implémentation

12 octobre 2026 — Version 0.2

Moteur Rust, bindings Python, R, WASM et MATLAB ; page Web avec composants nirs4all-ui. Conception revue, sans implémentation lancée.

Spécification de nirs4all preflight

Version 0.2 — 12 octobre 2026 — Conception revue par Astra et Opus, sans implémentation démarrée.

`nirs4all-preflight` qualifie un dataset NIRS avant entraînement. Il aide à décider si les mesures sont exploitables, quel domaine est couvert, quels problèmes vérifier et quelle validation choisir. Le moteur et les nouveaux diagnostics sont écrits en Rust. Une seule implémentation alimente Rust, Python, R, WASM et MATLAB ; la page navigateur compose les composants partagés de `nirs4all-ui`.

La roadmap définit les lots et gates. L'audit d'intégration, la review Codex et la review initiale Opus motivent les décisions. Le backlog de travail de l'écosystème reste `../../_backlogs/BACKLOG_CONSOLIDE.md` ; la présente roadmap détaille les features proposées, sans créer un deuxième registre de statut.

1 Objectifs et périmètre

Le public principal est un analyste NIRS disposant de spectres, de références éventuellement partielles et de métadonnées. Un utilisateur expert doit pouvoir auditer chaque résultat, retrouver le protocole et reproduire le calcul dans son langage. Aucune sortie ne promet à elle seule qu'une calibration atteindra une performance donnée.

Le parcours comporte six questions : les données sont-elles techniquement valides ; les acquisitions sont-elles répétables ; les domaines X et Y sont-ils couverts ; quels facteurs structurent les données ; les voisins spectraux sont-ils cohérents avec les références ; quelle validation répond à l'usage visé ? La synthèse conserve ces dimensions et des constats hiérarchisés. Elle ne calcule pas de note globale sur 100.

Livrable	Périmètre
MVP utile	Intégrité, identités et groupes, répétabilité absolue, bruit indirect, couverture X/Y, contradictions locales, associations métadonnées–Y, recommandations de validation, export JSON/HTML et page web.
Version 1 qualifiée	MVP, sonde PLS/PCR groupée, clusters avec stabilité, confounding multivarié, domain shift, bindings installables dans les cinq environnements.
Extensions	AOM supervisé, ICC déséquilibré avancé, mélanges de Y, visualisation UMAP/t-SNE et réseaux spectraux.
Hors périmètre	Calibration de production, active learning/HPLC, surveillance des modèles, remplacement de Quality/Studio, parsers propriétaires, backend de calcul Python, prise en charge générale des images ou séries ragged.

L'entrée V1 est une ou plusieurs sources spectrales denses, chacune avec son propre axe ; les analyses se font par source et par cible. Une jointure de sources n'est pas implicite. Les objets N-D ou ragged reçoivent un refus de profil explicite ; un hôte peut proposer un extrait dense via IO en conservant sa provenance. Les cibles continues et catégorielles ont des protocoles distincts. Censure, arrondi, limites de détection et précision de référence doivent être transportés lorsque disponibles.

2 Architecture et responsabilités

Couche	Propriétaire	Responsabilité
Décodage des fichiers	`nirs4all-formats`	Records, axes et provenance instrumentale ; support mémoire/WASM par format.
Dataset	`nirs4all-io` / `dag-ml-data`	Assemblage, identités, alignement, partitions, groupes et plan de données.
Qualification	`nirs4all-preflight`	Protocoles Rust, nouvelles statistiques propres au diagnostic, règles, contrats, orchestration des analyses et résultats.
Méthodes existantes	`nirs4all-methods`	PCA/SVD, prétraitements, T^2/Q, PCR/PLS et AOM via API publique ; extensions de diagnostics dans ce dépôt.

Couche	Propriétaire	Responsabilité
Validation ML	`dag-ml`	Plans de folds, sélection, exécution de sonde par phase et fold ; pas de second moteur CV dans les bindings.
Agrégat	`nirs4all-core`	Réexports, compatibilité et packaging du domaine `qualification` ; aucune nouvelle statistique.
R produit	`nirs4all-r`	Façade `nirs4all::qualify()` au-dessus du binding amont.
Présentation	`nirs4all-ui`	Contrats de vues et composants sans état applicatif, IO ou runtime.
Page web	`nirs4all-web/web-app`	Import, configuration, worker, persistance facultative et téléchargement de rapports.
Autres hôtes	Python complet, Studio, Quality	Adaptation de leurs datasets et restitution ; aucune copie du moteur.

Cette répartition maintient la doctrine de réutilisation tout en respectant Rust pour le nouveau moteur. Les diagnostics nouveaux, comme la répétabilité, les tests d'association et la stabilité de partitions, ont leur source de vérité ici. Une méthode générique déjà disponible en Methods est consommée via fournisseur, jamais recodée en Rust ou TypeScript pour contourner une capacité absente. Si une future statistique devient une méthode partagée, un transfert explicite de propriété doit précéder sa duplication.

Le workspace proposé contient `crates/nirs4all-preflight-core` pour les données, statistiques et règles pures ; `crates/nirs4all-preflight` pour l'API d'analyse et ses adaptateurs ; `crates/nirs4all-preflight-capi` pour l'ABI ; `crates/nirs4all-preflight-cli` pour l'outil en ligne de commande ; et `bindings/{python,r,wasm,matlab}`. Les bindings utilisant un outillage spécifique sont exclus du workspace principal, comme dans Formats/IO. Le core n'accède ni au filesystem ni au réseau. Les dépendances intra-workspace ont `path` et `version`, une édition Rust 2021 et des versions synchronisées.

`nirs4all-quality` demeure le produit de laboratoire. Il pourra consommer ce moteur pour sa phase amont ; sa calibration, sélection de références et maintenance restent indépendantes. Le terme `qualification` est utilisé dans les façades publiques pour éviter la confusion avec les contrôles d'installation/pipeline déjà nommés preflight.

3 Données et unités statistiques

L'adaptateur consomme un dataset déjà assemblé. Il construit un `QualificationInput` sans réinterpréter les joins ou conventions IO. Chaque source transporte `source_id`, `observation_ids`, `sample_ids`, matrice X, masque de validité, axe, unités, type de signal et provenance. Chaque cible transporte `target_id`, type, valeurs, masque, unités et identifiants de référence. Les métadonnées sont typées ; aucune chaîne numérique n'est convertie silencieusement.

Une observation est une acquisition. Un `physical_sample_id` est l'objet physique sur lequel peuvent exister plusieurs acquisitions. Un `reference_id` désigne une mesure de laboratoire. Les clés de lot, site, instrument, campagne, parent d'augmentation ou individu décrivent des dépendances ou domaines de généralisation ; elles ne sont pas substituées automatiquement à l'identité physique.

Champ	Exigence et comportement
Identités	L'alignement explicite par identifiant est obligatoire ; l'égalité du nombre de lignes ne suffit pas. Les acquisitions répétées ont des observation_ids distincts.
Identité physique inconnue	Intégrité et descriptions restent possibles. L'hôte peut déclarer « une acquisition par objet » ; cette hypothèse est enregistrée. Les inférences indépendantes restent limitées tant qu'elle n'est pas justifiée.
Répétitions	Type déclaré : rescan, repositionnement, re-préparation, biologique ou inconnu. Les groupes mixtes ne sont pas combinés en une seule erreur de mesure.
Groupes	Hiérarchie et facteurs croisés conservés. Le profil de validation choisit explicitement l'unité indépendante et la clé tenue à l'écart.

Champ	Exigence et comportement
Références	Une valeur de Y peut être commune aux acquisitions d'un sample ; ses répétitions labo restent distinctes. Pas de moyenne implicite de références contradictoires.
Sources partielles	Analyse source par source sur l'intersection explicite de samples et du masque de la cible ; publication du dénominateur et des exclus.
Augmentations	Les acquisitions générées sont exclues par défaut de la qualification du dataset réel et des OOF ; provenance parent conservée.
Partitions	Train/développement, test réservé et prédiction sans Y restent distincts. Aucun FIT sur le test réservé.

Le rapport publie `n_observations`, `n_physical_samples`, `n_reference_measurements`, `n_independent_units` et `n_validation_groups`, avec les hypothèses. Le champ `n_effective` reste nullable et ne reçoit une valeur que si un estimateur nommé est réellement calculé. Compter les samples n'est pas une correction universelle de corrélation intra-lot.

Les diagnostics de mesure utilisent les acquisitions natives. Les diagnostics de structure utilisent, par défaut, la moyenne des acquisitions techniques d'un même sample dans un contexte source × instrument × campagne homogène, avec poids égal par sample. Les contextes différents restent séparés ou utilisent un protocole explicitement multicontexte ; aucune moyenne ne gomme un effet instrument. L'agrégation et le dénominateur sont inscrits dans chaque résultat.

Avant toute agrégation, le plan construit les composantes d'identités indivisibles pour l'analyse : objet physique, acquisitions liées, parents d'augmentation et individu partagé lorsque pertinent. Il distingue ces liens d'identité des facteurs instrument/site servant à décrire le domaine. Les intersections entre composantes FIT et EVAL sont vérifiées sur les partitions d'origine, puis dans chaque niveau de CV. Les facteurs croisés ne sont pas supposés emboîtés : un individu mesuré sur deux sites peut rendre un holdout de site incompatible. Le moteur refuse ce plan ou demande un profil explicitement adapté ; il ne supprime pas des lignes en silence. Une agrégation ne mélange jamais acquisitions des deux partitions. Les hashes de lignes disjoints complètent ce contrôle mais ne le remplacent pas.

Pour X–Y, une référence composite partagée entre plusieurs objets crée aussi une dépendance de référence, transportée par `reference_group_id` et prise en compte dans les folds et permutations. Les mesures exactes individuelles sont admises. Cibles censurées/intervalle et références composites sans protocole adapté sont exclues des calculs exigeant des valeurs ponctuelles, avec effectifs et domaine d'interprétation restants. Aucun remplacement par LOD/2, borne ou duplication de référence en valeurs indépendantes. Des références incompatibles d'un sample demandent une décision de référence traçable avant calcul X–Y ; la qualification de X reste disponible.

Le plan déclare `independence_unit` (sample, reference_group, groupe nommé ou unknown), distinct des clés de domaine. Les voisins excluent par défaut cette unité entière ; inclure des voisins du même groupe constitue une sensibilité étiquetée. Avec unité unknown, couvertures et contrastes sont descriptifs, leurs tests sont `not_identifiable` et la sonde inférentielle est indisponible. Une hypothèse d'unicité déclenche un contrôle de répétitions présumées : quasi-identité spectrale et indices concordants de référence, acquisition ou métadonnées. Rapporter seuil/protocole, taux de paires suspectes et fraction d'observations concernées ; ce contrôle ne découvre pas une identité certaine. Tant qu'une suspicion ou l'hypothèse d'indépendance n'est pas résolue, confiance unknown ou low, sans p-value ou sonde présentée comme validée.

Les effectifs de répétition r_i et leur dispersion sont publiés. Une moyenne de plusieurs acquisitions peut être moins bruitée qu'une acquisition unique ; les profils de structure proposent une sensibilité à nombre de répétitions équilibré, avec sous-échantillonnage déclaré et seed. L'absence de répétitions d'un sample ne conduit pas à supprimer ce sample pour fabriquer artificiellement un plan équilibré.

4 Vues spectrales et garde contre la fuite

Trois vues sont nommées sans ambiguïté : `native`, `canonical:<protocol>` et `supervised:<target>:<protocol>`. La vue native conserve le signal mesuré. La vue canonique applique une recette non supervisée fixe, déclarée avant inspection de Y ; le profil initial utilise le centrage des variables, sans autoscaling ni whitening. Une sensibilité SNV optionnelle est étiquetée car elle peut supprimer de l'information d'amplitude. Les masques et conversions d'axe sont conservés ; aucune interpolation ou conversion reflectance/absorbance ne se fait sans provenance et paramètres.

La vue supervisée contient X transformé par le prétraitement sélectionné pour une cible, pas les scores PLS. Elle est désactivée dans le MVP et ajoutée après qualification d'AOM. Une vue refittée sur tout le développement est descriptive : clusters et cohérence locale y sont exploratoires, jamais une preuve indépendante de relation X–Y.

Pour toute mesure de performance, un plan externe groupé tient à l'écart des unités indépendantes. Chaque entraînement externe contient une sélection interne groupée des prétraitements et du nombre de composantes. Imputation, centrage, PCA, références MSC et autres états sont appris dans l'entraînement approprié puis appliqués aux lignes tenues à l'écart. Une sonde à recette fixée avant analyse n'exige pas de boucle de sélection des prétraitements, mais toute sélection de composantes reste interne. Les géométries apprises séparément par fold ne sont pas concaténées en un unique espace de clustering.

Le test réservé n'alimente ni sélection, ni seuil, ni recommandation. Une comparaison volontaire du domaine test aux données de développement peut lire X_test dans un profil `external_domain_audit`, sans apprentissage dessus. L'affichage de Y_test requiert un profil d'audit après évaluation ; le rapport marque alors le test comme consulté. Une décision de modèle ou de collecte prise après cette consultation nécessite une autre évaluation indépendante. Les hashes des populations FIT et EVAL, leurs intersections, les folds et le statut du test sont des artefacts obligatoires.

5 Protocoles analytiques

Les règles ci-dessous sont des protocoles proposés à versionner et calibrer sur fixtures. Un résultat manquant ne devient pas zéro et une absence de preuve ne devient pas une preuve d'absence. Une erreur d'alignement ou de mémoire peut bloquer une exécution ; une valeur statistique isolée ne bloque pas une calibration.

5.1 Intégrité

Contrôler dimensions, overflow, masque, finitude, identités, couverture, doublons et incompatibilité des unités. Un axe spectral connu doit être fini, ordonné et sans doublons ; un axe décroissant peut être renversé avec toutes ses colonnes et une transformation tracée. Les index sans unité permettent les descriptions matricielles mais pas les interprétations physiques de bandes. Les sources dont les grilles diffèrent ne sont pas concaténées silencieusement.

Les NaN/Inf sont rapportés comme anomalies ; le moteur n'apprend pas une imputation par défaut. Chaque protocole déclare s'il accepte un masque, un sous-ensemble complet ou exige des données complètes. La cause de l'exclusion et le nombre de samples restant sont rendus. Les doublons exacts sont calculés après normalisation documentée des floats et du masque ; un doublon technique reconnu n'est pas une erreur. Les quasi-doublons sont testés dans une vue et un contexte déclarés.

5.2 Répétabilité et qualité de mesure

Pour une bande λ et des groupes techniques homogènes ayant au moins deux acquisitions, la variance de répétabilité poolée vaut $s_r^2(\lambda) = \Sigma_i \Sigma_j (x_ij(\lambda) - \bar{x}_i(\lambda))^2 / \Sigma_i (r_i(\lambda) - 1)$. Les effectifs disponibles sont propres à λ. `s_r` est exprimé dans l'unité native ; le poids par degré de liberté, les `r_i` et la part de samples répétés sont publiés. Une dispersion robuste intra-sample complète cette estimation, avec une formule séparée et versionnée. Les intervalles sont obtenus en rééchantillonnant l'unité indépendante déclarée ; avec trop peu d'unités, le point descriptif reste accompagné de `insufficient_data` pour l'incertitude.

Un modèle $x_ij = \mu + u_i + \varepsilon_ij$ donne, si ses hypothèses sont pertinentes, $ICC = \sigma_sample^2 / (\sigma_sample^2 + \sigma_repeat^2)$. Le profil initial est l'ICC unidirectionnel de mesure unique, ICC(1,1), équilibré après application du masque propre à λ : $ICC_hat = (MS_between - MS_within) / [MS_between + (r-1) MS_within]$, avec MS les carrés moyens ANOVA, r répétitions et n samples complets. Il ne mesure pas la fiabilité d'une moyenne de r acquisitions. Les plans déséquilibrés reçoivent la répétabilité poolée, et ICC `unsupported`. Variance totale nulle donne `undefined` ; un estimateur ANOVA négatif reste visible et n'est pas ramené à zéro. REML déséquilibré, effets croisés et intervalles associés forment une extension ; les variances contraintes déclarent leur frontière. Un ICC élevé reste relatif à la population et peut refléter diffusion/ligne de base.

Le ratio de dispersions inter/intra est nommé « discrimination relative à la répétabilité » et ne reçoit pas le label SNR. Un SNR instrumental exige un protocole séparé avec bruit et signal de référence instrumentaux ; il n'est pas livré par défaut. Sans répétitions techniques, un résidu de lissage ou une énergie de différences finies est un `noise_proxy` : axe, fenêtre physique, normalisation et comportement aux bords sont déclarés. Le lissage existant passe par Methods.

Saturation n'est confirmée que si les bornes de l'instrument et l'unité sont connues. Une accumulation aux extrema observés est une suspicion de clipping. Le stitching est confirmé seulement sur une jonction connue ; une rupture détectée ailleurs peut être une bande chimique. La carte par λ conserve ses composantes (missingness, répétabilité, suspicion de clipping, rupture), leur disponibilité et leur incertitude, sans moyenne cachée en score de fiabilité.

5.3 Dimension et couverture de X

Une PCA du fournisseur Methods est apprise sur les samples de développement dans la vue canonique. Les valeurs propres utilisent la convention de variance n−1. Le profil initial choisit une compression modérée `d = min(10, n_samples−2, rang_numérique)` ; n<3 ou rang nul entraîne un résultat indisponible. `d_95` reste un diagnostic, pas le seul sélecteur de géométrie. La fraction expliquée réellement retenue est publiée. Les diagnostics de stabilité comparent d à `d_95`, 5 et 20 lorsque ces dimensions sont admissibles, sans choisir celle donnant le meilleur accord avec Y. Une première composante expliquant au moins 90 % déclenche un constat descriptif `first_component_dominant`, sans l'attribuer automatiquement à une nuisance. Une sensibilité SNV non supervisée est requise avant une conclusion forte de structure/coverage sensible à l'amplitude. Les seuils et cette règle de dimension restent des heuristiques à qualifier dans PF-00/03 ; avant cette preuve, confiance unknown. L'analyse parallèle naïve par permutation de colonnes n'est pas le défaut car elle détruit l'autocorrélation spectrale ; un nul adapté ou CV-PCA exige son propre protocole.

`D_eff = (Σ_a λ_a)² / Σ_a λ_a²` utilise les moments complets, même si les composantes PCA sont tronquées. Ils peuvent provenir du fournisseur ou être calculés exactement en Rust : pour X centré, `Σλ = ‖Xc‖²_F/(n−1)` et `Σλ² = ‖Xc Xcᵀ‖²_F/(n−1)²`. Le calcul de Gram se fait par blocs avec budget d'opérations explicite ; ne pas matérialiser n² doubles par défaut. Si ni ces moments ni le spectre complet ne sont disponibles dans le budget, `D_eff` est `unsupported`, jamais une dimension exacte issue des seules composantes retenues. `d_90`, `d_95`, `d_99` ont aussi un état d'absence si le spectre qualifié ne permet pas de les déterminer. Un helper interne ne constitue pas une API ; scores/loadings/résidus nécessaires restent des extensions publiques de Methods.

La couverture combine distances à un centre déclaré, T²/leverage lorsqu'ils sont qualifiés, résidus Q et distances aux voisins dans l'espace PCA. Les voisins excluent par défaut la même unité indépendante déclarée, dont toutes les acquisitions du même sample. Les classes descriptives « central, frontière, isolé, extrême » correspondent à des quantiles internes versionnés ; elles ne sont pas des classes de qualité ni une probabilité OOD. Les tailles de voisinage, normalisations et taux de données ignorées sont affichés.

5.4 Cibles et références

Pour chaque cible continue : effectif de samples référencés, quantiles, MAD, IQR, étendue, trous entre quantiles/valeurs ordonnées, valeurs censurées et distribution par métadonnée. Des répétitions de laboratoire fournissent une répétabilité de référence séparée. Le coefficient de variation est absent lorsque le niveau moyen est proche de zéro ; une normalisation par IQR vaut `undefined` si l'IQR est nul.

Pour une cible catégorielle : vocabulaire fixé, effectif indépendant par classe et groupe, classes absentes de folds et matrice classe × domaine. Les métriques de régression ne s'y appliquent pas. Cibles manquantes ou constantes permettent la qualification de X mais désactivent les diagnostics X–Y correspondants.

La multimodalité et les mélanges GMM sont des extensions : BIC/ICL, type de distribution, arrondi/censure, régularisation, convergence et stabilité doivent être explicites. Une GMM sur une concentration bornée ou des références discrétisées ne prouve pas plusieurs populations physiques.

5.5 Cohérence locale et contradictions

Le profil initial utilise `k = min(10, max(3, floor(sqrt(n_eligible))))`, borné par le nombre de voisins indépendants disponibles ; s'il reste moins de trois voisins, le résultat est indisponible. Les distances sont euclidiennes sur les scores PCA non blanchis. Les voisins sont choisis sans Y dans la vue canonique. Une sensibilité de k est rapportée dans le mode expert.

Pour Y continu, la statistique par sample est `L_i = 1 − mean_{j∈N_i}[(y_i−y_j)²] / (2 s_y²)`, avec variance globale des samples éligibles et poids égal par sample. Elle peut être négative

et n'est pas tronquée en score 0–1. Les valeurs de référence et leur précision accompagnent la détection de contradictions. Pour Y catégoriel, rapporter la fraction de voisins de même classe et son écart au niveau attendu avec les proportions des classes. Aucun des deux résultats n'est une performance de modèle.

La calibration compare une statistique globale préspécifiée aux permutations de Y autorisées par le plan ; les alertes locales sont exploratoires ou soumises à un contrôle de multiplicité. Une permutation de Y se fait au niveau du sample/groupe, jamais acquisition par acquisition. Si Y est constant dans chaque bloc autorisé et ne peut être permuté, le test est `not_identifiable`. Les doublons spectraux avec Y incompatible déclenchent « vérifier référence/appariement », sans suppression automatique. La tolérance de quasi-identité et celle de contradiction sont déclarées ; une précision labo absente ne devient pas une tolérance arbitraire.

Le contraste local brut n'est pas comparable directement entre samples : son niveau nul conditionnel dépend de y_i et peut être plus bas pour une référence extrême, même sans relation spectrale. Le MVP publie sa synthèse globale préspécifiée et des différences de référence avec distances spectrales ; il ne déduit pas de contradiction d'un seuil commun sur L_i. Un test local ultérieur maintient la référence focale et construit le nul conditionnel sur les voisins admissibles selon le plan qualifié. Pour une cible catégorielle, le niveau attendu utilise les proportions de classes dans la population de voisins admissibles, sans auto-inclusion.

5.6 Sonde de prédictibilité

PLS/PCR viennent de Methods et l'exécution de validation de DAG-ML. Le profil régression teste une grille bornée de composantes sur les entraînements internes, avec folds groupés explicites et budget de fits. La baseline est la moyenne de Y calculée dans le train de chaque fold externe. Publier prédictions OOF, RMSE sample-pondérée, MAE et R^2 global sur OOF lorsque son dénominateur est non nul, puis l'écart à la baseline et la variabilité par domaine. Les répétitions d'un sample ne créent pas plusieurs votes dans le score.

Une sonde de classification n'est activée qu'après qualification d'un estimateur et de ses probabilités/labels sur tous les hôtes concernés. Balanced accuracy est la moyenne des recalls des classes évaluables selon un vocabulaire fixe ; folds sans classes nécessaires sont signalés et peuvent rendre le profil inapplicable. Un split aléatoire et un split de domaine peuvent être comparés à titre de sensibilité, mais le choix du plus flatteur n'est pas une recommandation.

Une CV externe de 5 folds et interne de 3 constitue un objectif quand les groupes le permettent ; avec moins de groupes, le profil fixe un plan valide plus petit ou un refus. Les intervalles ne traitent pas les folds comme observations indépendantes. La sonde décrit le signal accessible à une famille limitée de modèles ; une sonde faible ne conclut pas que toute calibration est impossible.

Le profil initial agrège les répétitions techniques homogènes avant fit/predict, sans traverser une partition, et produit une prédiction OOF par sample ; aucune moyenne implicite de prédictions sur des contextes différents. Deux scores distincts sont publiés : $R^2_OOF = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y}_eval)^2$, définition descriptive standard ; et $skill_vs_train_mean = 1 - SSE_model / SSE_fold_train_mean$, où la baseline prédit chaque fold avec sa seule moyenne d'entraînement. Calculer $\bar{y}_eval$ pour normaliser le score d'évaluation n'apprend rien au modèle et ne constitue pas une fuite. Baseline et modèle ont exactement les mêmes samples et poids ; dénominateur nul donne undefined.

5.7 Clustering robuste

L'algorithme principal envisagé est HDBSCAN sur PCA modérée, après un gate comparant une implémentation Rust candidate à une référence. Semantique de `min_samples`, MST, sélection des clusters, gestion des ex aequo et score d'appartenance sont figées. Une implémentation candidate sans persistance ou sans sémantique compatible ne reçoit pas le label de parité HDBSCAN. Ward/GMM sont optionnels ; le MVP ne dépend pas de leur disponibilité.

La stabilité refait agrégation autorisée, centrage, PCA et clustering sur 50 sous-échantillons sans remise de 80 % des unités indépendantes. Les identités communes portent la comparaison. Rapporter ARI sur les samples assignés dans les deux partitions, accord assigné/bruit séparé, taille de l'intersection, fraction comparée et fréquence d'apparition de chaque groupe. Avec moins de deux clusters ou trop peu d'identités communes, ARI est indisponible. Le score d'appartenance HDBSCAN n'est pas une probabilité statistique de groupe réel.

Le protocole fixe `min_cluster_size` dans le jeu complet et le réplique `ceil(0.8 × min_cluster_size)` dans chaque sous-échantillon, sous la borne minimale admise par l'implémentation ; `min_`

samples suit la même règle déclarée. Cette politique de mise à l'échelle, l'arrondi et les cas de tailles trop faibles sont figés dans le golden et comparés à la variante à paramètres constants, sans sélection par Y.

Un modèle nul unimodal gaussien avec covariance régularisée dans l'espace PCA, sous les mêmes effectifs et protocole, sert de comparaison exploratoire ; il ne couvre pas toutes les structures continues possibles. Le nombre de clusters n'est annoncé fermement que si stabilité, séparation et taille minimale satisfont des règles calibrées. Sinon : « partition incertaine » ou « données compatibles avec une structure continue ». Les structures natives/canoniques/supervisées sont comparées sur les mêmes samples et accompagnées des taux de bruit. Leur différence suggère une sensibilité au prétraitement, sans attribuer automatiquement la cause à une nuisance ou à la chimie.

La confiance d'une partition dépend aussi de la fraction d'identités effectivement comparée ; un excellent ARI sur un petit sous-ensemble ne donne pas high. La fréquence d'apparition d'un cluster utilise un appariement par overlap/Jaccard sur identités communes, avec affectation univoque, seuil et gestion des égalités versionnés. Si l'appariement n'est pas qualifié, seules les distributions du nombre de groupes, de bruit et d'ARI global sont rendues.

5.8 Métadonnées et confounding

Le MVP montre tables croisées, couverture de Y par instrument/site/lot, tailles indépendantes et associations univariées adaptées au type, sans langage causal. L'objectif est d'identifier où une validation aléatoire ne couvre pas l'usage annoncé.

La version avancée utilise variance multivariée et PERMANOVA sur des distances euclidiennes PCA, avec effets marginaux, rang du design, facteurs aliassés et diagnostic de dispersion PERMDISP. Les R^2 marginaux ne sont pas additionnés comme parts exclusives. Le choix de permutation dépend de l'effet testé : permuter des labels de sample, des blocs entiers ou des résidus sous modèle réduit selon un protocole qualifié. Restreindre les permutations à un lot où l'instrument est constant ne teste pas l'effet instrument ; il faut des unités de domaine échangeables et suffisamment répétées. Un plan instrument=site=année peut rendre les effets séparés non identifiables.

Les explications de clusters sont descriptives. La prédictibilité de métadonnées depuis X se mesure sous validation groupée et se compare à une baseline ; elle n'établit pas que la métadonnée cause le signal. Les recommandations se fondent aussi sur les associations métadonnées–Y, l'usage prévu et la couverture des croisements.

La clé de validation tient à l'écart le niveau supérieur de dépendance pertinent (lot/campagne/individu) selon le plan, en vérifiant que les classes de la métadonnée restent évaluables. Si une classe instrument n'existe que dans un unique lot indépendant, sa prédictibilité hors lot n'est pas identifiable, plutôt que qualifiée par un split de samples du même lot.

5.9 Domain shift

Une comparaison de campagnes/instruments ou splits utilise un espace appris sur la population de référence puis appliqué à la population comparée. Le rapport sépare déplacement de centre, dispersion, couverture et changement de Y, avec unités et masques constants. Energy/MMD et classifier two-sample sont des profils avancés ; bandwidth, normalisation, échanges de groupes et sélection du classifieur sont déclarés et qualifiés. Un écart spectral ne prouve pas une baisse de performance et une absence de détection ne garantit pas l'équivalence.

Les seuils OOD appris sur développement s'appliquent sans refit aux nouveaux samples. Ils déclarent s'ils sont quantiles empiriques, tests théoriques ou heuristiques ; ils ne sont pas des probabilités d'échec. Le test réservé n'est jamais utilisé pour optimiser le bandwidth ou la règle d'alerte.

5.10 Réseaux spectraux

Le module expérimental utilise des bandes/λ comme nœuds et des corrélations entre samples indépendants comme arêtes, par source et vue. Il déclare signe, pondération, seuil, grille/bands, exclusion de liens adjacents et algorithme de communautés. Densité 2m/[p(p−1)], centralisation, modularité et transitivité sont accompagnées de sensibilité au seuil et de rééchantillonnages de samples. Graphe vide, isolats et dénominateurs nuls ont une sémantique explicite.

La publication de Ray et al. fournit une motivation pour décrire une architecture de corrélations dans des populations végétales. Le présent produit ne transforme pas cette approche en score de

qualité et n'adopte pas les effectifs ou seuils de l'étude comme minima génériques. Ce module ne conditionne aucune release V1.

6 Contrat de résultat et moteur de règles

Un `QualificationReport` porte `schema_version`, `engine_version`, `protocol_bundle_version`, `capabilities`, identité/hashes du dataset, populations, vues, paramètres, résultats, constats, recommandations et assets. JSON Schema versionne chaque enveloppe et vérifie les discriminants. Le résultat peut être `complete`, `partial`, `failed` ou `cancelled` ; les modules terminés d'un rapport partiel restent consultables mais la synthèse ne cache pas ceux qui manquent.

Objet	Champs essentiels
Résultat scientifique	`metric_id`, `protocol_version`, `status`, `reason_code`, `scope`, `source_id`, `target_id`, `view_id`, `population_id`, valeur et unité, dénominateurs, incertitude, paramètres, artefacts.
Disponibilité	`computed`, `not_applicable`, `insufficient_data`, `unsupported`, `not_identifiable`, `undefined`, `cancelled`, `error`. `value=null` hors computed, raison obligatoire.
Incertitude	Méthode, unité de rééchantillonnage, niveau nominal, bornes, réplicats valides/total et hypothèses ; nullable si non estimable.
Constat	ID de règle/version, observations/résultats justificatifs, sévérité, niveau de confiance, conséquence, action, limites, échantillons concernés.
Preuve de calcul	Version backend/ABI, seed et RNG, folds, populations FIT/EVAL, recette, masques/agrégation, durée, pic mémoire et approximation.
Asset	Table/array/graphique avec schéma, unités, description, shape, référence et hash ; pas de HTML exécutable dans le JSON scientifique.

Le JSON scientifique conserve nombres f64, pas NaN/Inf, valeurs nulles explicites et clés canoniques ; les grandes séries sortent en buffers ou fichiers référencés. La sérialisation structurelle est identique entre bindings ; la parité des nombres utilise des tolérances définies par protocole. Les différences de temps et d'environnement ne font pas partie du golden numérique.

Les niveaux de confiance `low`, `medium`, `high`, `unknown` décrivent la preuve et les conditions ; ils ne sont pas une probabilité. Les règles versionnées dérivent ces niveaux de la stabilité, disponibilité, effectifs et conformité aux hypothèses. Le seuil initial est une heuristique à valider, visible dans la vue experte. Seuls des défauts techniques certains peuvent être `blocking` ; les autres constats sont `info`, `warning` ou `high`. Aucun échantillon ou λ n'est retiré automatiquement.

Les p-values de permutation utilisent `(1+b)/(1+B)` pour les tirages Monte Carlo ; les tests exhaustifs ont leur propre dénominateur et convention. Les familles d'hypothèses sont enregistrées. Les tests confirmatoires préspécifiés utilisent Holm à 5 % ; la vue exploratoire peut publier Benjamini–Hochberg séparément et annoncer ses hypothèses de dépendance. Aucune étoile de significativité seule ne produit une confiance élevée. Les seeds de modules sont dérivées de manière stable d'une seed globale, du module, de la source et de la cible, avec RNG/version figés.

Le RNG des statistiques de qualification est implémenté dans le core Rust, algorithme/version figés, indépendamment de Python/R/JS. La racine u64 et les seeds DAG/n4m sont transmises sans arrondi JS, avec domaine de dérivation séparé pour chaque fournisseur ; une même seed ne prouve pas que deux RNG différents génèrent les mêmes tirages. Les plans/tirages nécessaires aux comparaisons sont conservés quand la reproductibilité exacte l'exige.

Chaque test transporte une `PermutationPlan` : hypothèse nulle et effet testé, unité échangeable, restrictions, tailles de blocs, modèle réduit éventuel, méthode de tirage, nombre de permutations distinctes admissibles, nombre exécuté et résolution. Le MVP admet une permutation uniforme de labels au niveau sample seulement si l'hypothèse d'échangeabilité des samples est déclarée ; les plans restreints/croisés doivent disposer d'un protocole qualifié. Il ne déplace pas automatiquement des blocs de tailles différentes et ne prend pas PERMDISP comme validation de l'échangeabilité. Budget initial : 999 permutations Monte Carlo par test global, révisable ; si la résolution ne permet pas de soutenir un seuil après multiplicité, aucune conclusion de significativité n'est produite. Les analyses descriptives ne sont pas interdites par un plan de permutation indisponible.

Chaque lot qualifie ses règles dès sa livraison, avec fixtures et version de protocole ; PF-09 étend cette preuve au corpus de RC. Avant validation d'une règle, confiance `unknown`, constat exploratoire et aucune classe affirmée comme fiable. Taille élevée et p-value faible ne suffisent

pas. La recommandation de validation déclare domaine de généralisation visé, unités retenues, couverture et limites d'identifiabilité.

Un exemple de résultat d'absence est : `metric_id="measurement.icc"`, `status="unsupported"`, `reason_code="UNBALANCED_REPLICATES_UNSUPPORTED"`, `value=null`, tandis que `measurement.repeatability_sd` peut être computed. Ce comportement doit être identique dans tous les bindings.

7 API et bindings

L'API Rust cible `capabilities()`, `validate_input()`, `plan()`, `analyze()` et un job incrémental `step()/cancel()/report()`. Le plan expose modules, populations, coûts estimés et capacités requises avant allocation lourde. Une façade de haut niveau accepte les objets IO ; une API matrice accepte X/Y avec identités et options explicites. Les tableaux sans identités ne contournent pas les règles d'alignement.

L'ABI publique proposée utilise le préfixe `n4pf_*`, une version indépendante des packages, un contexte opaque et des vues de matrices f64 avec shape, strides, longueur de buffer et masque. Un handle n'est utilisable que dans son contexte. Créer/détruire contexte, valider l'entrée, créer le job, step/cancel, récupérer JSON/table/buffer, libérer le résultat, lire l'erreur et vérifier la compatibilité couvrent la surface initiale. Les noms exacts et snapshots sont figés au lot 0.

Les wrappers vérifient nullité, alignement, longueurs, shapes et overflow des offsets. Une ABI C ne peut pas prouver qu'un pointeur arbitraire appartient à une allocation valide : cette validité, la durée de vie et l'absence de mutation concurrente sont des préconditions obligatoires de l'appelant. Les wrappers sûrs les garantissent avant création de la vue. Le support initial couvre strides positifs et layouts ligne/colonne ; strides négatifs et dtype non f64 imposent une copie explicite ou un refus. Les bindings conservent les objets propriétaires pendant les appels. Les buffers sont empruntés pour un appel synchrone ; un job incrémental copie ses données ou détient un propriétaire dont la durée de vie est explicitement attachée au job. Aucun job n'emprunte un tableau temporaire. Les handles invalidés ne sont pas réutilisables ; après panic récupéré, le contexte passe en erreur et doit être détruit, puis recréé.

Le contexte contient ses erreurs et son fournisseur Methods ; l'ABI retourne un statut et ne fait traverser ni panic Rust ni exception C++. Les allocations core sont libérées par le core qui les a créées ; les buffers fournis par l'appelant restent à lui. `catch_unwind` est appliqué aux frontières quand la stratégie de panic le permet ; erreurs d'allocation et panics non récupérables sont documentés. Un contexte par job/thread évite le partage non sûr du backend n4m. Les callbacks n'appellent jamais le runtime R/MATLAB depuis un thread de travail.

Binding amont proposé	Façade utilisateur	Particularités
Python `nirs4all-preflight`, import `nirs4all_preflight`	`nirs4all_core.qualification`, adaptateur optionnel du Python complet	NumPy, masques, GIL libéré, typage ; aucune dépendance runtime obligatoire à la bibliothèque Python complète.
R `nirs4allpreflight`	`nirs4all::qualify()` dans `nirs4all-r`	R/.Call vers C ABI, matrices colonne-major, NA explicites, finalizers ; éviter une dépendance obligatoire à Python.
npm `@nirs4all/preflight-wasm`	Domaine `qualification` de l'agrégat `nirs4all`	TypeScript typé, TypedArrays, jobs incrémentaux, module WASM versionné ; aucun calcul scientifique JS.
MATLAB + `nirs4allpreflight`	`nirs4all.qualify()` dans Core	Dispatcher MEX fin vers ABI C, tableaux réels double colonne-major, tables/strings adaptés ; erreurs MATLAB cohérentes.
Rust crate `nirs4all-preflight`	Domaine de la crate agrégée `nirs4all`	Types sûrs et API bas niveau sans chargement des bindings hôtes.

Ces noms sont des propositions à vérifier dans les registres avant publication. Le binding Python peut utiliser PyO3 pour la conversion mais appelle la même façade native ; R utilise une passerelle C stable, sans deuxième implémentation extendr des statistiques. Les distributions contiennent ou déclarent leurs dépendances natives selon le profil, avec ABI et licences vérifiables. Python/R/MATLAB importent un rapport même si une méthode optionnelle n'est pas exécutable.

La distribution R vise d'abord GitHub/R-universe et un build source avec crates vendorisées, Cargo/rustc de versions déclarées et outillage de plateforme ; Rtools seul ne fournit pas la chaîne Rust. Un binaire peut être fourni par la matrice de build, avec provenance et même ABI. Les MEX Linux doivent tester les interactions avec la libstdc++ embarquée par MATLAB et la dépendance C++ de Methods ; l'ABI C n'élimine pas un conflit du chargeur. Liaison/visibility et bibliothèque retenue sont qualifiées sur un vrai MATLAB avant release.

La matrice de qualification est par opération × version × langage × OS × profil. `available`, `qualified`, `experimental` et `unavailable` sont distingués ; un module compilé n'est pas automatiquement qualifié. MATLAB doit être exécuté sous licence sur une version déclarée, avec MEX, layouts et cycles d'allocation. Octave est un gate complémentaire, jamais un substitut à cette preuve. La version 1 « cinq bindings » reste incomplète tant que ce gate MATLAB manque.

La matrice suivante est une cible de livraison, entièrement non implémentée à cette phase. « Tous » inclut Rust, Python, R, WASM et MATLAB ; aucune case ne constitue une disponibilité actuelle.

Opération	Jalon cible	Environnements visés	Condition supplémentaire
Validation, intégrité et rapport	PF-01/02	Tous	ABI et mêmes erreurs/statuts.
Répétabilité et noise proxy	PF-02	Tous	Recette Methods disponible pour le proxy qui l'utilise.
PCA, dimension et couverture	PF-03	Tous	Fournisseur et getters PCA qualifiés ; spectre complet ou résultat limité déclaré.
Cibles, cohérence et associations descriptives	PF-03	Tous	Références admissibles et identités.
Sonde régression groupée	PF-04	Tous	Vrai DAG/Methods dans l'hôte ; navigateur peut rester partiel si son gate échoue.
Clusters, confounding et shift	PF-05/06	Tous	Profil de données, permutations et budget qualifiés.
AOM, ICC REML et mélanges Y	PF-10	Selon capacités qualifiées	Aucun repli caché dans un binding.
Réseaux	PF-11	Profil expérimental	Non requis pour V1.

PF-00 fixe les versions de Python 3.11+, R, Node, compilateurs Rust/C et MATLAB réellement testées. Les profils natifs visent Linux x64, Windows x64 et macOS arm64 ; l'accès aux runners et dépendances de chaque surface est confirmé avant engagement. Le navigateur vise Chromium, Firefox et WebKit avec versions consignées ; tests Node/WASM complètent mais ne remplacent pas les navigateurs. Une cible non exécutée reste explicitement non qualifiée.

8 Fournisseurs numériques et WASM

Un `MethodsProvider` expose les opérations publiques nécessaires à PCA, lissage, prétraitements et diagnostics. Le `ValidationProvider` reçoit la spécification complète de sonde, le plan de données/validation et le budget ; DAG-ML possède toute la boucle externe/interne, sélection et phases, puis renvoie OOF, scores, folds et populations. Preflight ne pilote pas lui-même une seconde boucle CV et ne traite pas DAG comme simple splitter. Les appels Methods de la sonde passent par les contrôleurs DAG qualifiés. Sur CPU, les diagnostics Rust appellent n4m par ABI C via un adaptateur versionné. Le core peut produire les analyses qui n'exigent pas ces fournisseurs sans charger leur bibliothèque ; les autres sont `unsupported`, sans fallback scientifique caché.

Dans le navigateur, Preflight est un module Rust `wasm32-unknown-unknown`, Methods/libn4m un module Emscripten, et DAG-ML possède son propre module Rust/wasm-bindgen ; DAG-ML n'est pas assimilé à Emscripten. Le worker transporte requêtes, tableaux et résultats entre les mémoires, sans implémenter PCA, PLS ou règles statistiques. Rust décide les modules de diagnostic et demande une sonde complète au fournisseur DAG. PF-00 teste d'abord une importation synchrone dans le même worker pour les diagnostics Methods : wasm-bindgen → adaptateur JS de transport → export Emscripten, sans ASYNCIFY. La cohérence de runtime, propriété et erreurs doit être démontrée ; cette option n'est pas présumée fonctionner.

Le repli asynchrone `step → provider_request → provider_response → step` n'est activé que par une seule machine à états Rust partagée CPU/WASM, avec adaptateur CPU qui la fait avancer jusqu'à terminaison. La réponse est liée à job, request_id, hash des entrées, phase et fold ; une réponse étrangère/périmée est rejetée. Il ne doit pas exister deux algorithmes de protocole ni deux boucles CV. PF-00 fige l'option qui ferme ses gates ; le transport de sonde suit les capacités réelles DAG et reste distinct du callback de diagnostic.

L'échange de matrices implique des copies mesurées ; il ne promet pas de partage de pointeurs entre modules. La liaison directe d'un Rust wasm-bindgen avec Emscripten n'est pas supposée fonctionner. Une alternative de build unifié `wasm32-unknown-emscripten` ne sera choisie qu'après preuve, sans modifier les algorithmes ni créer une dépendance browser dans le core.

Chaque hôte déclare un résolveur de dépendances natives : préférer l'instance Methods déjà chargée par l'agrégat/paquet amont, vérifier version/ABI et ne jamais échanger des handles entre deux instances. Dans une page/worker, Core et Preflight partagent une seule instance Emscripten n4m. Si un package autonome embarque statiquement une copie privée, ses symboles sont cachés, son fournisseur/ABI distincts et aucun handle ne traverse cette frontière. La politique de liaison, les hashes et les conflits au chargement font partie de la matrice de capacités et du packaging.

Le spike du lot 0 doit exécuter une petite chaîne IO → matrice → Methods PCA → résultat Rust, dans Node et un vrai navigateur, avec résultats comparés au CPU, pic mémoire, messages asynchrones et annulation. Une dépendance n4m absente réduit les capacités annoncées ; elle n'active pas une PCA JS. Si les méthodes lourdes ne sont pas qualifiées, la page livre le MVP calculable et une démo explicitement bornée, en affichant les modules indisponibles et la commande native pour le dataset complet.

Un deuxième spike exécute une sonde minimale à recette fixée, avec deux folds groupés et vrai train/transform/predict DAG-ML sur CPU et navigateur. Le `ValidationProvider` possède les plans, phases et handles d'exécution DAG ; le `MethodsProvider` possède les états numériques dont il reçoit le fit. Rust décrit les requêtes et collecte les résultats ; l'hôte n'invente ni folds ni phases. Tout état porte backend, job, source, population FIT, phase et fold ; aucune réponse tardive après annulation ni aucun état d'un autre fold n'est accepté. Ce spike ne qualifie pas à lui seul la recherche AOM imbriquée, réservée à PF-10.

Le checkout DAG-ML contient déjà un adaptateur natif Methods et des contrôleurs runtime WASM. Cela ne prouve pas leur couverture de la sonde imbriquée ciblée : PF-00 établit la tranche réutilisable et les extensions requises, avec charge amont. PF-04 ne démarre pas sa qualification tant que le contrôleur estimateur et les phases nécessaires ne sont pas réellement livrés.

9 Page web et composants partagés

La page de qualification est une route de `nirs4all-web/web-app`, accessible dès qu'un dataset est assemblé. Le parcours est : importer via IO/Formats ; confirmer identité, répétitions, sources/cibles, groupes et objectif de généralisation ; consulter le plan ; lancer l'analyse ; parcourir les constats ; exporter. Les métadonnées manquantes donnent des questions de configuration et des états d'absence explicites, sans bloquer la description de X.

L'analyse reste locale au navigateur ; aucun téléversement automatique de spectres. La persistance IndexedDB est facultative, avec suppression accessible. Le worker restitue progression, erreurs et résultats partiels ; Cancel détruit le worker ou utilise le signal natif, puis libère les modules. Le rapport contient les versions exactes et le profil complet/partiel. Les jeux de démo sont redistribuables, décrivent leurs anomalies et portent le label « démonstration » dans chaque export.

Chaque module terminé émet un snapshot immuable avec sa provenance avant de lancer le suivant. L'hôte conserve ces snapshots ; une destruction du worker reconstruit un rapport `cancelled` avec seulement les résultats confirmés, sans faire passer le module interrompu en computed. Les buffers/handles dépendant de la mémoire détruite ne sont jamais réutilisés. Des messages terminés après Cancel sont ignorés via la génération du job.

Réutilisation vérifiée	Usage
`DatasetPreviewCard`, `StepProgress`	Population et progression.
`SpectraPlot`, `Histogram`, `PcaScatter`, `BoxPlot`	Spectres, références et structure déjà calculés.

Réutilisation vérifiée	Usage
`FeatureImportanceBar`, `FoldStabilityLines`, `ScoreHeatmap`	Effets et stabilité lorsqu'ils sont disponibles.
`HealthFindingRow`, `DecisionCard`, badges runtime	Constats et état d'exécution, via adaptation conservant sévérité et disponibilité.

Créer dans `nirs4all-ui` un domaine de présentation `qualification` avec `QualificationSummary`, `QualificationFindingCard`, `WavelengthReliabilityPanel`, `CoveragePanel`, `ClusterStabilityPanel`, `ValidationAdvice` et `MetricEvidenceTable` lorsque les contrats existants sont insuffisants. Les props fournissent données, descriptions, textes localisés et actions de l'hôte. Ni API scientifique, ni route, ni worker, ni IndexedDB dans UI. Les textes sont FR/EN ; les IDs scientifiques restent indépendants de la langue.

Chaque graphique répond à une question, affiche unités/population/vue et dispose d'un résumé visible et d'une table filtrable synchronisée. CSV complète les grandes tables mais ne remplace pas le résumé HTML. Les états ne reposent pas sur la seule couleur ; clavier, focus, zoom 400 %, 320 pixels, contraste, reduced motion et lecteur d'écran sont des gates. Les composants SVG existants fournissent des descriptions mais l'hôte doit ajouter les données structurées utiles. Export HTML conserve texte et tables ; PDF conserve les conclusions et les valeurs nécessaires, avec qualification d'accessibilité séparée.

Les composants de qualification exigent une prop `profileStatus` (complete, partial, demo, cancelled) et la liste des modules indisponibles. Ce marquage reste visible dans les cartes et exports ; un jeu démonstratif ou profil partiel ne prend pas le titre d'une qualification complète.

10 Budgets et performances

Les profils initiaux sont des budgets à mesurer, pas des performances promises. Petit navigateur : jusqu'à 500 samples indépendants × 2 000 λ × une source/une cible. Profil étendu visé : 2 000 × 4 000 × deux sources, sous budgets mémoire et opérations explicites. Le seul X f64 de 2 000 × 4 000 occupe 64 Mo décimaux ; quatre copies, vues et temporaires peuvent dépasser 256 Mo avant clustering. La planification comptabilise ces copies et les buffers des deux modules.

Le plafond proposé du worker est 256 Mio pour le petit profil, 512 Mio pour le profil étendu après mesure. Clustering all-pairs est interdit au-delà d'un n qualifié ; kNN par blocs et index exact/approximatif déclarés, budgets de sous-échantillons, de permutations et de fits sont requis. Les approximations produisent un marqueur de protocole et une preuve d'écart au calcul exact. Une limite déclenche un résultat partiel/refus explicite, jamais un sous-échantillonnage silencieux de l'intégrité.

Le budget mémoire inclut la somme des high-water marks des mémoires linéaires Rust, DAG et Emscripten, plus les TypedArrays JS et sorties conservées. Libérer un buffer ne fait pas rétrécir la mémoire linéaire existante. Les snapshots permettent de terminer/recréer le worker entre phases lourdes quand le plan le prévoit ; cette stratégie ne conserve pas des handles devenus invalides. Le profil étendu reste non qualifié tant que le pic cumulé n'a pas été mesuré. Les 512 Mio sont un plafond de refus et non une promesse que le profil y tient.

Objectifs initiaux à confirmer : intégrité/description <2 s, MVP sans rééchantillonnage lourd <15 s sur le petit profil, progression observable et annulation <1 s entre blocs, hors appel fournisseur non interruptible. Un fournisseur non interruptible est annulé par destruction du worker dans le navigateur. La sonde et les modules avancés annoncent leur budget, sans promesse de durée fixe. Les temps sont mesurés sur une machine et des navigateurs publiés avec p50/p95 et pic mémoire. Aucun thread WASM partagé ou header COOP/COEP n'est requis dans le premier build Pages.

11 Validation et livraison

Les gates Rust sont `cargo fmt --all --check`, `cargo clippy --workspace --all-targets -- -D warnings`, `cargo test --workspace`, rustdoc et licences. Les tests d'ABI vérifient symboles/version/ownership, erreurs et layouts ; sanitizers/fuzzing ciblent les frontières et entrées adversariales. Pas d'accès à des données privées dans CI.

Les fixtures synthétiques isolent corruption d'alignement, répétitions inégales, bruit connu, rang contrôlé, cible constante/partielle/censurée, contradictions, confounding exact, shift, continuum et clusters instables. Les tests d'invariants contrôlent changement d'ordre des lignes/colonnes, conversion de layout, réplication technique sans gain artificiel et absence de fuite. Des fixtures

réelles publiques apportent une validation contextualisée avec source, licence et SHA, sans prétendre à des seuils universels.

Chaque protocole possède une référence pertinente : formules analytiques pour répétabilité ; outils R dédiés pour composantes de variance et PERMANOVA ; hdbscan pour partitions ; Python complet/Methods pour PCA/PLS/prétraitements. Les références sont test-only et leurs licences n'entrent pas implicitement dans le runtime. Les goldens sont générés avec versions, provenance et tolérances ; aucune mise à jour pour faire passer une divergence inexpliquée. PCA se compare par variance, projecteurs et reconstruction, pas par signe brut de vecteurs ; clustering par équivalence des partitions et gestion des ex aequo.

Toutes les surfaces exécutent les mêmes cas via leurs packages installés, y compris données column-major R/MATLAB et erreurs typées. Les manifests, capability matrices et docs sont générés à partir des preuves de tests. Le build web se fait avec des packages épinglés distribuables, sans chemin local de frère ni WASM copié à la main. UI exécute typecheck/build/Vitest et rendu SSR ; Web ajoute worker, accessibilité et smoke navigateur ; Studio suit ses gates lint/test si intégré.

La licence proposée suit `CECILL-2.1 OR AGPL-3.0-or-later` des couches Rust/UI actuelles, sous vérification des dépendances et du packaging. Fournir notices, provenance de copies, SBOM et hashes. Ne pas revendiquer CRAN, App Store ou certification normative ; aucune publication n'est réalisée par la présente phase de rédaction.

12 Décisions et limites restantes

Sont fixés pour lancer le travail : core Rust pur ; trois vues nommées ; unités explicites ; contrat commun et états d'absence ; réutilisation de Methods/DAG ; worker à deux modules ; page Web et composants UI ; MVP indépendant du clustering ; absence de score global et de suppression automatique.

Le lot 0 ferme les décisions dépendantes d'un test : cohorte publique compatible, API de diagnostic PCA, transport asynchrone, outillage MEX et accès MATLAB, disponibilité Rust HDBSCAN, coûts des copies. Si un spike échoue, la roadmap réduit un profil en le rendant visible ; elle ne change pas silencieusement de langage ou d'algorithme. Les réseaux restent hors calendrier de V1. Les reviews finales et leur traitement sont enregistrés dans les décisions de review.

Roadmap d'implémentation de nirs4all preflight

12 octobre 2026 — Lots proposés, aucune implémentation démarrée. La spécification définit le comportement cible ; le backlog consolidé `../../_backlogs/BACKLOG_CONSOLIDE.md` conserve le statut des features. Les identifiants PF servent à relier détails de conception, futures issues et preuves de qualification.

Le chemin critique est contrat/identités → ABI et passerelle WASM → diagnostics utiles → sonde validée → qualification multi-langages. UI progresse dès les premières sorties ; AOM et réseaux ne bloquent pas le MVP. Les charges sont des jours-personnes d'ingénierie, incluant tests et docs de chaque lot, pour des contributeurs connaissant l'écosystème. Elles restent à réestimer après les spikes.

1 Lots et critères d'acceptation

Lot	Charge	Dépendances	Livrables et gate de sortie
PF-00 Contrats et faisabilité	10–14 j	Aucune	Schémas input/report/capabilities, composantes d'identités indivisibles, matrice réutilisation/manque, cohorte épinglée. Spikes IO→PCA→Rust et vrai DAG train/transform/predict à deux folds groupés CPU/navigateur ; callbacks synchrones/repli unifié, copies, liaison et annulation ; compilation R/MEX et accès MATLAB confirmés.
PF-01 Socle Rust et bindings verticaux	16–22 j	PF-00	Workspace, C ABI versionnée, CLI, packages Python/R/WASM/MATLAB et validation d'entrée. Le même petit dataset passe validate/analyze/report dans chaque hôte ; erreurs, strides, ownership, libération, outillage R/Cargo et import sans Python complet testés.
PF-02 Intégrité, mesure et restitution initiale	12–18 j	PF-01	Intégrité, unités, répétabilité absolue, ICC(1,1) équilibré qualifié, masque/λ et noise proxy ; JSON/HTML ; première page Web, contrats/components UI. Bruit connu et répétitions inégales ; aucun SNR inventé ; cancel/snapshots partiels.
PF-03 Couverture et cohérence X–Y	10–14 j	PF-02 + API PCA de PF-00	PCA canonique, dimension, couverture X/Y, références partielles, contradictions/kNN hors répétitions, associations métadonnées–Y et recommandations. Politiques censure/composites et permutations explicites ; règles qualifiées ou confiance unknown ; nul local conditionnel ; parité CPU/WASM. **MVP utile.**
PF-04 Sonde et validation groupée	16–24 j	PF-03 + fournisseurs Methods/DAG qualifiés	Sonde PLS/PCR, baseline, OOF par sample, plans groupés et sélection de composantes interne. Tests de fuite et dataset à batch confondu ; mêmes folds et score dans chaque langage ; état indisponible si plan impossible.
PF-05 Clusters et stabilité	12–18 j	PF-03 ; spike HDBSCAN PF-00	Implémentation Rust qualifiée, paramètres sémantiques fixés, sous-échantillonnages sans remise, bruit/intersections/ARI, sensibilité PCA et modèle nul. Continuum et partitions fragiles rendus incertains ; aucune obligation de trouver des groupes.
PF-06 Confounding et domain shift	10–16 j	PF-04 ; PF-05 pour explication des clusters	Effets marginaux, design/alias, PERMDISP et permutations valides ; comparaison de domaines dans un espace appris sur référence. Tests contre oracles et exemples non identifiables ; recommandation liée à l'usage et à métadonnées–Y.

Lot	Charge	Dépendances	Livrables et gate de sortie
PF-07 Produits et composants complets	14–22 j	PF-02 en continu ; PF-03→06 pour données finales	Page Web complète dans le profil qualifié, composants UI partageables, FR/EN, vue experte, rapports, adaptation pilote Studio/Quality sans migration générale. Clavier/lecteur d'écran, tables équivalentes, persistance opt-in, dépendances npm épinglées sans chemin sibling.
PF-08 Packaging et intégration des façades	8–12 j	PF-01 en continu ; PF-04→07 pour matrice finale	Wheels, package R, npm/WASM, Rust et MATLAB/MEX installables ; domaine Core et façade R ; symboles/versions/notices/SBOM. Tests depuis distributions installées, pas uniquement checkout ; qualification MATLAB réelle distincte d'Octave.
PF-09 Qualification scientifique et RC	8–14 j	PF-02→08	Campagne synthétique + corpus public, seuils/règles versionnés, parité cinq surfaces, budgets navigateur, rapports reproductibles, docs et changelog. Aucune anomalie P0/P1 sans résolution ou exclusion explicite du profil ; dossier RC concret avant publication. **Version 1.**
PF-10 Extensions ciblées	24–40 j, optionnel	PF-09	AOM imbriqué avec stabilité d'opérateurs, ICC REML déséquilibré, mixtures Y qualifiées et visualization non linéaire. Activer séparément chaque capacité avec témoins de calcul ; estimation à détailler après V1.
PF-11 Réseaux spectraux	Hors estimation V1	PF-09 + objectif scientifique validé	Prototype descriptif avec sensibilité au seuil/autocorrélation, bootstrap de samples et validation de son intérêt utilisateur. Décision de poursuivre fondée sur un benchmark, sans score de calibrabilité.

PF-05 et PF-06 peuvent être décalés après une première release MVP s'ils retardent la valeur utile. Ils restent requis pour le périmètre V1 décrit ; un décalage ne transforme pas une release partielle en V1 complète.

2 Jalons et organisation

La somme PF-00 à PF-09 est **116–174 jours-personnes**, après réestimation des spikes et cinq bindings à la suite des reviews. Les adaptations amont additionnelles ci-dessous ajoutent **8–22 jours-personnes**, soit une enveloppe V1 de **124–196 jours-personnes avant réserve**. La durée calendaire dépend du chemin critique, des propriétaires amont et de l'accès MATLAB ; elle n'est pas le quotient automatique de la charge par deux contributeurs. PF-10 ajoute 24–40 jours-personnes ; PF-11 est une exploration distincte.

Cette enveloppe couvre les tâches Preflight et les adaptations bornées d'interface, pas la résolution d'une dette amont de grande ampleur. J0 confirme charges, releases, disponibilités des propriétaires et matrice minimale OS × langage × opération. Ajouter ensuite une réserve de 20–30 % si les spikes restent incertains ; elle n'est pas incluse dans les sommes précédentes. Les tâches des différents propriétaires ne deviennent pas parallèles du seul fait que deux personnes travaillent sur Preflight.

À périmètre inchangé, appliquer 20 % au bas et 30 % au haut donne environ **149–255 jours-personnes**, hors dette amont supplémentaire. À deux personnes disponibles, un ordre de grandeur calendaire de **18–30 semaines** tient compte d'intégration et de dépendances ; il reste à recalculer à J0 et n'engage aucune date. Le MVP PF-00→03 représente **48–68 jours-personnes**, avant adaptations amont requises et réserve, avec une valeur utile plus tôt que V1.

Jalon	Contenu livrable	Condition de passage
J0, fin PF-00	Contrats et décision navigateur	Spike réel, API/propriétaires clarifiés, risques chiffrés ; réduction de profil si nécessaire.
J1, fin PF-01	Squelette exécutable cinq surfaces	Entrée et rapport identiques, erreurs et ownership, binding MEX réel.

Jalon	Contenu livrable	Condition de passage
J2, fin PF-03	MVP utile CPU + Web	Mesure/couverture/cohérence et recommandations ; limites visibles ; présentation et exports.
J3, fin PF-04	Sonde de signal	Aucune fuite démontrée sur fixtures adversariales ; folds/OOF inspectables.
J4, fin PF-05/06	Diagnostics de structure	Stabilité, incertitude et non-identifiabilité correctement rendues.
J5, fin PF-07→09	RC puis V1	Packages installés et cinq bindings qualifiés ; matrices et budgets publiés.

La phase documentation actuelle s'arrête avant PF-00. Lors de l'implémentation, le responsable moteur porte contrats/statistiques/CLI et ABI ; le responsable produit porte adaptateurs UI, page Web et intégration hôtes. Chaque lot touchant Methods, IO, DAG, Core ou R reçoit une issue/PR dans son dépôt propriétaire, avec dépendances explicites. Les modifications ne sont pas regroupées comme si les dépôts étaient un monorepo.

Une interface de fournisseur peut être simulée pour tester le protocole Rust mais le mock ne ferme jamais le gate d'exécution réel. Les tests end-to-end sont réalisés sur une cohorte de versions concrètes et conservés dans une preuve machine-readable. Les décisions de review ne modifient pas un statut en « livré » avant ces témoins.

3 Travail amont et dépendances

Projet	Demande prévue	Gate lié
Methods	Diagnostic PCA public : valeurs propres/moments complets, loadings/scores/résidus, convention numérique ; vérifier PLS/PCR, prétraitements et fold_ids AOM sur la cohorte.	PF-00/03/04/10 ; changements ABI avec snapshots/changelog.
IO et Data	Adapter le dataset assemblé au contrat de qualification, vérifier observation/sample/reference IDs, groupes, types de répétition et sources partielles. Les ajouts de données restent amont.	PF-00/01/03.
DAG-ML	Fournisseur de validation groupée, populations FIT/EVAL et OOF ; qualifier les phases internes/externes et le protocole demande/réponse.	PF-04 et PF-10.
UI	Réutiliser `viz/lab` et publier `qualification` pour les champs scientifiques supplémentaires ; contrats de présentation purs.	PF-02/07.

Projet	Demande prévue	Gate lié
Web	Ajouter route et worker adaptateur ; injecter modules Rust et Emscripten distribués, budget mémoire et état des modules manquants.	PF-00/02/07.
Core et R produit	Réexport `qualification`, façade R, registry de compatibilité et matrice par opération.	PF-08.
Studio et Quality	Pilote de consommation des rapports/composants ; pas de remplacement du parcours ni de migration générale.	PF-07, puis extension selon priorité.

Les dettes existantes QUA-02/03, MTH-10, NEW-01 et NEW-08 du backlog restent des dépendances ou réutilisations possibles, pas des tâches automatiquement absorbées. Un registre de capacités commun peut être consommé s'il existe ; sa réalisation globale n'est pas une condition pour un registre local honnête.

Adaptation amont bornée	Charge indicative	Comptabilisation et release
Methods : getters diagnostics PCA et exposition qualifiée	4–8 j	Additionnel ; version/ABI et release publique avant PF-03. Réutiliser les getters existants s'ils suffisent.
DAG-ML : tranche de sonde groupée et adaptateur vers contrôleurs Methods existants	4–10 j	Additionnel ; API existante à vérifier, pas hypothèse d'absence. Release avec phases requises avant PF-04.
IO/Data : lacunes de transport d'identités/références seulement	0–4 j	Additionnel si lacune constatée ; sinon zéro. Adaptateur Preflight déjà dans PF-01/03.
UI : contrats et composants partagés	4–8 j	Inclus dans PF-07 ; package UI épinglé avant packaging final, pas ajouté deux fois.
Core/R produit : réexports et façades	3–6 j	Inclus dans PF-08 ; releases compatibles dans la cohorte, pas ajouté deux fois.

Ces charges additionnelles totalisent 8–22 jours-personnes. Si une opération existe déjà, J0 réduit sa ligne avec preuve ; si une tranche de validation générale manque, elle constitue un nouveau périmètre à chiffrer plutôt qu'une extension cachée de PF-04.

4 Risques et décisions de repli

Risque	Détection précoce	Décision
Deux mémoires WASM trop coûteuses	PF-00 : copies, pic RSS/heap, job réel	Réduire le profil navigateur ou livrer démo/MVP ; analyses complètes via CLI/bindings natifs. Aucun calcul scientifique JS de remplacement.
API PCA incomplète	Vérification des headers/getters et test sur valeurs propres	Extension dans Methods ; description/intégrité livrables avant elle, couverture géométrique indisponible tant que non qualifiée.

Risque	Détection précoce	Décision
HDBSCAN Rust insuffisant	Fixtures de densités, bruit, ex aequo et persistance	Reporter PF-05 ; PCA/coverage et résumé « clustering indisponible ». Aucune imitation de persistance.
Identités ou domaines absents	PF-01 : validation et configuration	Hypothèses enregistrées ; analyses descriptives seulement ; ne pas prétendre à indépendance ou généralisabilité.
Trop peu de groupes ou confounding exact	PF-04/06 : plan et rang du design	Refus de la sonde concernée/non-identifiabilité, recommandation de compléter les croisements.
MATLAB non accessible sous licence	PF-00 puis PF-01	Préparer MEX/Octave et matrices ; garder MATLAB non qualifié et gate V1 cinq bindings ouvert. Pas de revendication de parité MATLAB.
Seuils d'alerte fragiles	PF-09 : corpus synthétique et réel	Déclasser en observation exploratoire ou confiance basse ; pas de blocage statistique ni de score agrégé.

5 Qualification et clôture des lots

Chaque lot produit : diff/PR dans le dépôt propriétaire, contrats et capacités mis à jour, tests appropriés, fixtures avec provenance, preuve d'exécution, documentation utilisateur et risques restants. Rust passe fmt/clippy/tests ; l'ABI vérifie version et symboles ; les bindings passent leurs tests depuis des installations isolées ; UI/Web passent leurs contrôles et smokes appropriés. Les comparaisons scientifiques contrôlent formules, invariants et références indépendantes, pas seulement une seconde exécution du même code.

Les seuils, règles et niveaux de confiance ont leur gate minimal dans le lot qui les expose. Les modules non calibrés sont exploratoires avec confiance unknown dès le MVP ; PF-09 étend la validation sans attendre pour corriger une règle qui produirait déjà une alerte trompeuse. Les familles de tests, budgets et résolution de permutations sont des contrats du module, pas une décision tardive de la RC.

La RC exige un jeu réel public et des cas adversariaux pour : repetitions inégales, référence manquante/constante, répétitions exclues des voisins et folds, fuite volontaire rejetée, plan instrument/site aliasé, continuum, bruit pur et décalage de domaine. Native-versus-binding et références-versus-moteur sont deux gates distincts. Les métriques aléatoires ont RNG, paramètres, tolérances et limites de stabilité déclarés.

La publication ultérieure passe par les pratiques de release des dépôts concernés. Cette roadmap prépare un résultat reviewable ; elle ne lance ni développement, ni commit, ni publication. La synthèse finale et le registre des reviews décrivent les décisions retenues et ce qui reste à prouver.

Audit de l’intégration de nirs4all preflight

État des checkouts locaux inspectés le 12 octobre 2026. Cet inventaire établit les frontières et les points d’intégration ; il ne qualifie pas les exécutables ni les distributions publiques. Les versions de catalogue et les symboles ne suffisent pas à prouver la disponibilité d’une opération dans un langage.

Sources et périmètre

Source principale : `qualification_nirs_architecture_plan.docx`, version du 14 septembre 2026, SHA-256 `7670941ce51943f37a8b92b7c1df16ca3b67b5b4970f1a96d41555853db77d1c`.

Le fichier voisin `audit_metriques.docx` concerne surtout des métriques de comparaison de modèles et un manuscrit. Il ne constitue pas un deuxième cahier des charges de Preflight. Son exigence de définitions, dénominateurs et seuils explicites est pertinente ; ses corrections de Nemenyi et de CRPS restent hors périmètre.

Les instructions racine et celles des dépôts pertinents sont prises en compte. Formats, IO, Methods et Web ont été interrogés via CodeGraph avant les recherches de code. Les exports UI, manifestes, catalogues et en-têtes publics complètent cet examen. Les répertoires privés Lab et Drafts sont exclus ; aucun contenu privé n’a été utilisé.

La vérification ciblée après review inclut aussi DAG-ML via CodeGraph. `crates/dag-ml-wasm/Cargo.toml` utilise wasm-bindgen, js-sys et wasm-bindgen-futures : DAG-ML est un module Rust/WASM, distinct du module Emscripten libn4m. `crates/dag-ml-wasm/src/initial_refit.rs` expose une exécution REFIT avec contrôleurs runtime et callback JS synchrone. L’index identifie `crates/dag-ml-core/src/methods_estimator.rs` ; une absence générale d’API estimateur Rust ne peut donc pas être déduite d’anciennes notes. La couverture exacte de la sonde imbriquée reste à qualifier au lot 0.

Responsabilités et réutilisation

Dépôt	Contrat constaté	Conséquence pour Preflight
`nirs4all-formats`	Workspace Rust 2021 ; records spectraux avec axes, dimensions, unités, type de signal et provenance ; lectures mémoire distinguées des lectures nécessitant le filesystem.	Réutiliser les lecteurs ; disponibilité par format et cible à vérifier.
`nirs4all-io`	`DatasetSpec`, sources, partitions et folds ; séparation core pur/façade IO ; assembleur sans filesystem utilisé en WASM ; contrats JSON canoniques.	Consommer les objets assemblés ; ne pas reconstruire joins, identités ou splits aléatoires dans le loader.
`dag-ml-data` et `dag-ml`	Couche de données et orchestration ML séparées ; agrégat et Web s’appuient sur ces contrats.	Consommer les plans de validation groupée et les exécutions de sonde ; auditer les capacités exactes au premier lot.
`nirs4all-methods`	Méthodes natives, ABI C, vues de matrices avec strides, wrappers multi-langages et WASM.	Réutiliser PCA, prétraitements, PLS et AOM existants via un fournisseur, avec versions et limites explicites.
`nirs4all-core`	Agrégat portable, sans nouvelles méthodes ni nouveaux loaders. R produit déplacé dans `nirs4all-r`.	Ajouter un domaine de qualification après livraison du package amont ; ne pas ajouter de moteur dans Core.

Dépôt	Contrat constaté	Conséquence pour Preflight
nirs4all-r	Produit utilisateur nirs4all, composant n4m/DAG/IO ; aucune réimplémentation numérique.	Prévoir un binding amont puis une façade produit R.
nirs4all-ui	Composants React sans état applicatif, IO, browser globals ou runtime ; domaines viz, lab, dataset, runtime.	Réutiliser graphiques et alertes ; ajouter des composants de qualification et leurs contrats de présentation.
nirs4all-web	Client React/Vite/WASM actif sous web-app/, worker avec progress/cancel, GitHub Pages.	Ajouter la page de qualification et son adaptateur worker dans le client actif.
nirs4all-quality	Produit guidé de laboratoire : re-mesure, sélection HPLC, calibration, prédiction et maintenance.	Consommateur possible du moteur et des composants ; préserver son parcours distinct.

Points de code confirmés

nirs4all-ui/src/viz/index.ts exporte notamment SpectraPlot, Histogram, PcaScatter, BoxPlot, FeatureImportanceBar, FoldStabilityLines et ScoreHeatmap. src/lab/index.ts exporte DecisionCard, HealthFindingRow, StepProgress et WorklistTable. Le contrat HealthFindingInput ne transporte pas encore toute la provenance, l'incertitude et les scopes nécessaires à la qualification scientifique. Son adaptation doit conserver ces champs dans la vue experte.

nirs4all-methods/cpp/include/n4m/decomposition.h expose n4m_decomposition_flexible_pca_* pour create/fit/transform/output-cols. Ces déclarations n'offrent pas d'accesseur dédié aux valeurs propres, loadings et résidus complets. Le helper interne cpp/src/core/common/pca_helper.c calcule scores et variances expliquées ; il ne doit pas être lié directement comme API publique. Une extension de diagnostic publique est donc un point de gate.

Le catalogue catalog/methods/aom_pop.aom_pls.yaml décrit n4m_model_selection_aom_pls_select, des getters de résultats et l'entrée fold_ids. Il contient aussi une tolérance de parité tbd et ne constitue pas une preuve de qualification. Une campagne ciblée doit vérifier les folds, la sélection de prétraitement et le transfert des états appris dans chaque environnement.

nirs4all-web/web-app/src/engine/worker-engine.ts fournit déjà le mécanisme de jobs, progression, refus typés et annulation par destruction du worker. La passerelle Rust/WASM de Preflight pourra s'appuyer sur ce mécanisme, mais devra traiter explicitement les copies et ownership entre les mémoires Preflight/wasm-bindgen, DAG/wasm-bindgen et libn4m/Emscripten.

Révisions locales inspectées

Dépôt	HEAD
Formats	8c6acd6b02b0bb746ae8df43efa062515ef4a774
IO	27c7cf064bb5e31b13b8b67f9c47a370b999d75d
Methods	1adf214c3c2204095e21a9ff1cdebf6faaf2d4d2
UI	db401cdcee0d60b102b0894310d6bb994701255a
Core	5668796aaac9a02d8d0146ec05ead04f9c76657c
R	1c6f4a7f1b42a5d550cabc4809e957680630a4d7
Web	0ec8790b4afc896a01c0f85ef59f9e347a83c3a2
Quality	6d68810b4b5961eb6a77f881f35f8cf7e33480a2
DAG-ML	c6b52979745397ae4e8662580df79341f0378d28
DAG-ML-Data	5ab19aec77fdca443c83a24a6b9d663c41a82b4a

Ces SHA servent de point de départ documentaire, pas de versions minimales de distribution. La

cohorte d’implémentation devra être épinglée sur des releases réellement installables et les tests exécutés à nouveau.

Références scientifiques externes

La documentation de validation imbriquée décrit le biais produit par une sélection et une évaluation sur les mêmes folds. Cela justifie une sonde avec sélection interne et évaluation externe distinctes.

La documentation HDBSCAN précise les limites en haute dimension. La prédiction de nouvelles affectations maintient une partition existante et ne correspond pas à un reclustering de l’ensemble augmenté. La politique de stabilité de Preflight devra choisir et déclarer le cas évalué.

La publication de Ray et al., publiée initialement le 17 août 2026, est bien identifiable. Elle présente des architectures de corrélations de longueurs d’onde dans une étude de populations végétales. Cette portée ne justifie pas des seuils génériques de calibrabilité ni un effectif minimal universel. Les valeurs de rarefaction mentionnées dans le DOCX ne sont donc pas reprises comme règles produit.

Les choix de modèle d’ICC et les interfaces MEX servent de références aux exigences correspondantes. Les protocoles retenus sont détaillés dans la spécification, avec leurs limites de support.

Décisions issues des reviews de nirs4all preflight

12 octobre 2026. Les avis originaux sont conservés dans `reviews/`. Les corrections ci-dessous sont intégrées à la spécification 0.2 et à la roadmap. Il s'agit d'une validation de conception ; les gates scientifiques, ABI et produit restent à exécuter pendant l'implémentation.

1 Review du document source

La review Codex et la review Opus ont été réalisées avant rédaction de la spécification. Opus a effectivement été exécuté via Claude Code CLI avec le modèle `claude-opus-5-5`. Son avis initial est livré sur deux pages A4. Le DOCX original est conservé sans modification, avec son SHA dans l'audit.

Finding	Décision intégrée
Pseudo-réplication	Observation, sample, référence et groupes distingués ; analyses de structure après agrégation homogène ; voisins/folds/rééchantillonnages respectent les unités. Pas de n effectif universel.
Fuite par AOM	Trois vues nommées ; vue supervisée descriptive ; sélection interne et évaluation externe ; AOM reporté après V1 ; géométries de folds non concaténées.
SNR/ICC	Répétabilité absolue prioritaire. Ratio inter/intra nommé discrimination relative ; SNR instrumental hors profil initial ; ICC modèle/plan/conditions déclarés.
Confounding	Associations métadonnées–Y dès le MVP ; effets marginaux/non-identifiables, dispersion et permutations adaptées ensuite. Aucune part additive ou causalité inventée.
Clusters/bootstrap	Sous-échantillonnage sans remise, comparaisons sur identités communes, bruit et fraction comparée. Profil PCA borné avec sensibilité ; analyse parallèle/CV-PCA possible ultérieurement.
Y, censure et mixtures	Couverture d'abord ; censure et références exactes distinguées ; GMM/multimodalité conditionnelles, hors MVP.
WASM et réutilisation	Spikes des deux mémoires et fournisseur asynchrone ; Methods/DAG conservés ; aucune statistique TypeScript ; page Web et composants UI.
MVP trop large	MVP utile PF-00→03 ; sonde PF-04 ; structure avancée PF-05/06 ; réseaux PF-11 hors estimation V1.

Trois avis sont adaptés avec justification. Les groupes physiques inconnus n'interdisent pas l'intégrité et les descriptions, mais limitent les inférences. Le terme SNR reste réservé à un éventuel protocole instrumental défini, plutôt qu'interdit comme concept. Les erreurs techniques certaines d'alignement ou de mémoire restent bloquantes avant calcul ; les seuils statistiques ne bloquent pas automatiquement une calibration. La sonde est livrée juste après le MVP pour éviter qu'une dépendance DAG retarde les diagnostics de mesure.

2 Review finale Astra

La review indépendante Astra porte sur le draft 0.1 écrit après réception de l'avis initial d'Opus. Son verdict est favorable au lancement du lot de faisabilité sous réserves ciblées. Tous ses findings sont traités dans le contrat 0.2 ; les preuves d'exécution correspondantes deviennent des gates, pas des réussites supposées.

Finding Astra	Traitement dans la spécification et la roadmap
P0-1 Étanchéité des identités liées	§3/4 : composantes indivisibles avant agrégation ; intersections physiques/parents/individus à chaque partition et niveau CV. Fixture de mêmes objets sous observation_ids distincts, PF-00/04.
P1-2 Permutations contractuelles	§6 : PermutationPlan avec hypothèse, unité, restrictions, modèle réduit, cardinalité/résolution et refus de schémas non qualifiés. Appliqué dès PF-03.
P1-3 Nul des contrastes locaux	§5.5 : contraste global préspécifié ; absence de seuil brut commun ; nul conditionnel pour test local ultérieur ; proportions catégorielles sur voisins admissibles.

Finding Astra	Traitement dans la spécification et la roadmap
P1-4 Censure et référence composite	§3 : valeurs exactes admises, exclusions explicites sans LOD/2, dépendance reference_group_id ; pas de duplication comme labels indépendants.
P1-5 Seuils tardifs	§6 et roadmap : gate scientifique dans chaque lot ; unknown/exploratoire avant calibration ; PF-09 élargit la preuve.
P1-6 Spike DAG/WASM	§8 et PF-00 : deuxième spike train/transform/predict réel à deux folds groupés ; ownership phases/états et refus de réponses tardives.
P1-7 Garantie des pointeurs	§7 : préconditions C explicites, wrappers responsables, pas de preuve impossible d'allocation ; durée de vie/mutation/handles/panic définis.
P2-8 ICC par bande	§5.2/6 : ICC(1,1) de mesure unique et ANOVA nommée ; équilibre après masque ; undefined/unsupported distingués ; valeurs négatives conservées.
P2-9 Clusters et annulation	§5.7 : confiance conditionnée à fraction comparée, appariement Jaccard versionné ; §9 : snapshots terminés gardés par l'hôte avant destruction.
P1-10 Estimation et dépendances	Roadmap §2 : dette amont séparée, réserve 20–30 %, matrice OS/langage et capacité des propriétaires à confirmer à J0.

3 Review finale Opus

La review finale Opus est une review documentaire du même draft 0.1. Aucun outil de dépôt n'était activé pour ce reviewer ; des intentions de commandes apparaissent dans sa session comme texte, sans exécution. Les notes de session qu'il cite sur DAG-ML ne sont pas une preuve de capacité actuelle. Le contrôle ciblé du principal confirme un module DAG Rust/wasm-bindgen et l'existence d'un adaptateur Methods ; leur couverture de sonde reste à qualifier.

Finding Opus	Décision intégrée
P0-1 Voisins/permutations et indépendance	Unité d'indépendance déclarée, exclusion par défaut de cette unité ; sensibilité intragroupe étiquetée ; tests non identifiables si unité unknown ; aucun échange automatique de blocs non qualifié.
P0-2 Répétitions inconnues	Contrôle obligatoire de répétitions présumées sous hypothèse d'unicité, taux de paires/observations suspectes et confiance limitée ; ne pas reconstruire des identités certaines à partir de Y identique.
P0-3 Pilote de sonde	DAG-ML possède toute la CV et la sélection ; ValidationProvider reçoit une sonde complète et rend OOF/folds/populations ; Preflight ne recrée pas la boucle. Gate et charge amont explicites.
P1-1 Transport synchrone/asynchrone	Spike callback synchrone dans le même worker en priorité ; repli via une seule machine à états Rust commune CPU/WASM ; pas de deux implémentations de protocole.
P1-2 Instances n4m	Résolveur par hôte, ABI/hash vérifiés ; instance Emscripten partagée dans la page ; copie privée statique isolée et interdiction d'échanger des handles.
P1-3 Dimension PCA	d95 n'est plus le seul sélecteur ; compression modérée bornée, sensibilités et dominante PC1 signalée. Une variance dominante ne prouve pas une nuisance. Horn par permutation naïve des colonnes n'est pas retenu par défaut car l'autocorrélation NIRS serait détruite.
P1-4 Résolution des permutations	Budget 999 initial par test global et plan avec cardinalité/résolution ; absence de conclusion si multiplicité et résolution l'empêchent. Ne pas augmenter automatiquement le budget sans borne.
P1-5 Mémoire WASM	Somme des high-water marks des modules et buffers JS ; snapshots/recréation de worker possibles ; profil étendu non qualifié avant mesure.
P1-6 Chiffrage	PF-00 10–14 j, PF-01 16–22 j ; coûts amont bornés additionnels 8–22 j, autres coûts inclus identifiés ; V1 124–196 j avant réserve, 149–255 avec réserve indicative.
P2 Dimension effective	Moments exacts via fournisseur ou calcul Rust par blocs, avec budget ; pas de D_eff exact à partir d'un spectre tronqué.
P2 Répétitions inégales	Dispersion des r_i et sensibilité équilibrée documentées ; pas de suppression silencieuse des samples non répétés.

Finding Opus	Décision intégrée
P2 R^2 et OOF	R^2 descriptif standard et skill_vs_train_mean distincts ; baseline apprise dans les trains ; une prédiction par sample après agrégation homogène.
P2 Clusters, métadonnées, RNG	Mise à l'échelle déclarée des paramètres de densité ; holdout de dépendance pertinent et classes évaluables ; RNG core figé, seeds fournisseurs exactes et distinctes.
P2 Bindings et MVP	R/Cargo/vendorisation et conflits libstdc++ MATLAB qualifiés ; ICC équilibré dans PF-02 ; permutation/descriptif explicités dès PF-03 ; profileStatus obligatoire dans UI.

Deux suggestions statistiques ne sont pas reprises comme assertions générales. La moyenne des Y d'évaluation dans le dénominateur du R^2 standard ne fuit pas vers le modèle : elle normalise son évaluation ; le gain face à une baseline de train est une autre métrique, fournie séparément. Le plafond proposé $1 - \sigma_ref^2/s_y^2$ pour un contraste local exige des hypothèses de bruit indépendant, homogène et de signal parfait ; ce n'est pas une borne universelle des observations. La précision de référence reste publiée et utilisée dans un protocole explicite, sans plafond présenté comme certain.

4 Limites de la clôture documentaire

Les reviews ne prouvent ni justesse de l'implémentation Rust, ni exécution dans les cinq langages, ni disponibilité MATLAB. Les points nécessitant des essais deviennent des critères d'acceptation explicites. La première action de développement est PF-00 ; aucune publication ou modification de code produit n'est réalisée dans cette phase.